

AI USAGE REPORT

Model used: ChatGPT

Most of the project was assisted by AI and I only made some small changes. I described what I want and put the prompt into chatGPT. Design ideas are mine.

Prompt: help with LevelSelect. Level Design: (5%) The scene should have “Static” wall The scene should include question blocks which can interact with the player

Answer:

Your **LevelSelect** and **Level Design** are two different scoring parts.

The PDF says **Level Select** is part of the game process, while **Level Design** requires the playable scene to include a **Static wall** and **question blocks that interact with the player**. 

So do it like this:

```
Start.scene
↓
LevelSelect.scene
↓
Game.scene
```



1. LevelSelect.scene

In `LevelSelect.scene`, make a simple UI:

```
Canvas
├─ titleLabel: "Select Level"
├─ Level1Button
└─ BackButton
```



Create a script called `LevelSelect.ts`:

```
<> TypeScript

import { _decorator, Component, director, sys } from 'cc';

const { ccclass } = _decorator;

@ccclass('LevelSelect')
export class LevelSelect extends Component {
  playLevel1() {
    sys.localStorage.setItem('selectedLevel', '1');
    director.loadScene('Game');
  }

  backToStart() {
    director.loadScene('Start');
  }
}
```



Attach this script to a node like:

```
LevelSelectManager
```



Then for the **Level1Button**:

```
Button component
→ Click Events
→ Drag LevelSelectManager into the empty slot
→ Select LevelSelect → playLevel1
```



For **BackButton**:

For **BackButton**:

```
Button component
+ Click Events
+ Drag LevelSelectManager
+ Select LevelSelect → backToStart
```

If your screen is still black, check that your `Canvas` is inside the scene and your `Camera` can see it.

2. Static wall in Game.scene

In `Game.scene`, create this:

```
World
├─ Ground
├─ LeftWall
├─ RightWall
└─ PlatformWall
```

For each wall or ground node, add:

```
Sprite
RigidBody2D
BoxCollider2D
```

Set the `RigidBody2D` type to:

```
Static
```

This satisfies:

```
The scene should have "Static" wall
```

Example setup:

```
Ground
Position: x = 0, y = -250
Scale or size: wide platform
RigidBody2D: Static
BoxCollider2D: same size as ground sprite
```

```
LeftWall
Position: x = -500, y = 0
RigidBody2D: Static
BoxCollider2D: tall vertical wall
```

3. Question block setup

Create a node:

```
World
├─ QuestionBlocks
└─ QuestionBlock
```

Add these components to `QuestionBlock`:

```
Sprite
RigidBody2D
BoxCollider2D
QuestionBlock script
```

Set:

Set the `RigidBody2D` type to:

Static

This satisfies:

The scene should have "Static" wall

Example setup:

Ground

Position: `x = 0, y = -250`

Scale or size: wide platform

`RigidBody2D`: Static

`BoxCollider2D`: same size as ground sprite

LeftWall

Position: `x = -500, y = 0`

`RigidBody2D`: Static

`BoxCollider2D`: tall vertical wall

3. Question block setup

Create a node:

```
World
├── QuestionBlocks
│   └── QuestionBlock
```

Add these components to `QuestionBlock`:

Sprite

`RigidBody2D`

`BoxCollider2D`

`QuestionBlock` script

Set:

`RigidBody2D` Type: Static

Create `QuestionBlock.ts`:

```
import {
  _decorator,
  Component,
  Collider2D,
  Contact2DType,
  IPhysics2DContact,
  Prefab,
  Instantiate,
  Sprite,
  SpriteFrame,
  Vec3
} from 'cc';

const { ccclass, property } = _decorator;

@ccclass('QuestionBlock')
export class QuestionBlock extends Component {
  @property(Prefab)
  mushroomPrefab: Prefab | null = null;

  @property(SpriteFrame)
  usedBlockSprite: SpriteFrame | null = null;

  private used: boolean = false;

  start() {
    const collider = this.getComponent(Collider2D);

    if (collider) {
      collider.on(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
    }
  }

  onDestroy() {
    const collider = this.getComponent(Collider2D);

    if (collider) {
      collider.off(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
    }
  }

  private onBeginContact(
    selfCollider: Collider2D,
    otherCollider: Collider2D,
    contact: IPhysics2DContact | null
  ) {
```



```

TypeScript

const playerY = otherCollider.node.worldPosition.y;
const blockY = this.node.worldPosition.y;

// Player must hit the block from below
if (playerY < blockY - 5) {
    this.activateBlock();
}
}

private activateBlock() {
    this.used = true;

    const sprite = this.getComponent(Sprite);
    if (sprite && this.usedBlockSprite) {
        sprite.spriteFrame = this.usedBlockSprite;
    }

    if (this.mushroomPrefab) {
        const mushroom = instantiate(this.mushroomPrefab);
        this.node.parent?.addChild(mushroom);

        const spawnPosition = this.node.worldPosition.clone();
        spawnPosition.y += 40;

        mushroom.setWorldPosition(spawnPosition);
    }
}
}

```

Then in the Inspector:

```

QuestionBlock script
├─ Mushroom Prefab: drag your Mushroom prefab here
└─ Used Block Sprite: drag your empty/used block sprite here

```

4. Mushroom makes player bigger

Create a `Mushroom` node or prefab.

Add:

```

Sprite
BoxCollider2D
RigidBody2D
Mushroom script

```

Set:

```

RigidBody2D Type: Static
BoxCollider2D Sensor: checked

```

Create `Mushroom.ts`:

```

TypeScript

import {
    _decorator,
    Component,
    Collider2D,
    Contact2DType,
    IPhysics2DContact,
    Vec3
} from 'cc';

const { ccclass } = _decorator;

@ccclass('Mushroom')
export class Mushroom extends Component {
    start() {
        const collider = this.getComponent(Collider2D);

        if (collider) {
            collider.on(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
        }
    }

    private onBeginContact(
        selfCollider: Collider2D,
        otherCollider: Collider2D,
        contact: IPhysics2DContact | null
    ) {
        if (otherCollider.node.name !== 'Player') return;

        otherCollider.node.setScale(new Vec3(1.4, 1.4, 1));

        this.node.destroy();
    }
}

```

Now the interaction is:

```
Player jumps from below
→ hits QuestionBlock
→ QuestionBlock changes to used block
→ Mushroom appears above block
→ Player touches Mushroom
→ Player becomes bigger
```

That should satisfy both:

```
The scene should include question blocks which can interact with the player
```

and:

```
At least 1 type of blocks, super mushroom which makes Mario bigger
```

 mario

For your README, you can write:

```
<#> Markdown
### Level Design
The game scene includes static walls and platforms using Static Rigidbody2D and BoxCollider2D. The sc
```

     ...  Sources

okay help me with the game scene first

Thought for 15s >

Okay, build **Game.scene** first. This scene should cover the assignment's **World Map**, **Level Design**, **Player**, **Question Block**, and later enemies/UI. The PDF specifically says the scene needs correct physics, camera/background movement, static walls, and interactive question blocks. 

I'm assuming **Cocos Creator 3.x + TypeScript**.

1. Game.scene node structure

In **Game.scene**, make this hierarchy:

```
Game.scene
├── Camera
├── Canvas
│   └── UI
│       ├── LifeLabel
│       ├── ScoreLabel
│       └── TimerLabel
├── World
│   ├── Background
│   ├── Ground
│   ├── LeftWall
│   ├── RightWall
│   ├── QuestionBlock
│   ├── Player
│   └── GameManager
```

For now, focus on:

```
World
├── Ground
├── LeftWall
├── RightWall
├── QuestionBlock
└── Player
```

Your black screen probably means your scene has no visible sprite/background yet, or the camera is not seeing the objects.

2. Set up the Camera

Select **Camera**.

Set position:

```
Position X: 0
Position Y: 0
Position Z: 1000
```

Set background color to something visible, like light blue.

If the preview is still black, check that the camera is enabled and exists in the scene.

3. Create Ground

Select `World`, then create an empty child called:

Ground

Add components:

Sprite
RigidBody2D
BoxCollider2D

Set:

RigidBody2D Type: Static

Set position and size:

Position X: 0
Position Y: -250
Width: 1200
Height: 50

For the `BoxCollider2D`, press **Editing** or adjust the size so it matches the ground.

This is your first **Static wall/platform**.

4. Create LeftWall and RightWall

Create `LeftWall`.

Add:

Sprite
RigidBody2D
BoxCollider2D

Set:

RigidBody2D Type: Static
Position X: -600
Position Y: 0
Width: 50
Height: 600

Create `RightWall`.

Same components, but:

Position X: 600
Position Y: 0
Width: 50
Height: 600

Now your level has static boundaries.

5. Create Player

Create a child under `World` called:

Player

Add:

Sprite
RigidBody2D
BoxCollider2D
PlayerController script

Set:

Position X: -300
Position Y: -150
RigidBody2D Type: Dynamic
Fixed Rotation: true

Also set the player node name exactly:

Player

This matters because the question block script will check for `Player`.

6. Player movement code

Create a script:



assets/scripts/PlayerController.ts



Paste this:

TypeScript



```
import {
  _decorator,
  Component,
  Input,
  Input,
  EventKeyboard,
  KeyCode,
  RigidBody2D,
  Vec2,
  Collider2D,
  Contact2DType,
  IPhysics2DContact
} from 'cc';

const { ccclass, property } = _decorator;

@ccclass('PlayerController')
export class PlayerController extends Component {
  @property
  moveSpeed: number = 250;

  @property
  jumpForce: number = 450;

  private rigidBody: RigidBody2D | null = null;
  private moveDir: number = 0;
  private isGrounded: boolean = false;

  start() {
    this.rigidBody = this.getComponent(RigidBody2D);

    const collider = this.getComponent(Collider2D);
    if (collider) {
      collider.on(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
      collider.on(Contact2DType.END_CONTACT, this.onEndContact, this);
    }

    input.on(Input.EventType.KEY_DOWN, this.onKeyDown, this);
    input.on(Input.EventType.KEY_UP, this.onKeyUp, this);
  }

  onDestroy() {
    input.off(Input.EventType.KEY_DOWN, this.onKeyDown, this);
    input.off(Input.EventType.KEY_UP, this.onKeyUp, this);
  }

  update() {
    if (!this.rigidBody) return;

    const velocity = this.rigidBody.linearVelocity;
    this.rigidBody.linearVelocity = new Vec2(this.moveDir * this.moveSpeed, velocity.y);
  }

  private onKeyDown(event: EventKeyboard) {
    if (event.keyCode === KeyCode.KEY_A || event.keyCode === KeyCode.ARROW_LEFT) {
      this.moveDir = -1;
    }

    if (event.keyCode === KeyCode.KEY_D || event.keyCode === KeyCode.ARROW_RIGHT) {
      this.moveDir = 1;
    }

    if (
      (event.keyCode === KeyCode.SPACE || event.keyCode === KeyCode.KEY_W || event.keyCode ===
      && this.isGrounded
    ) {
      this.jump();
    }
  }

  private onKeyUp(event: EventKeyboard) {
    if (
      event.keyCode === KeyCode.KEY_A ||
      event.keyCode === KeyCode.ARROW_LEFT ||
      event.keyCode === KeyCode.KEY_D ||
      event.keyCode === KeyCode.ARROW_RIGHT
    ) {
      this.moveDir = 0;
    }
  }

  private jump() {
    if (!this.rigidBody) return;

    const velocity = this.rigidBody.linearVelocity;
    this.rigidBody.linearVelocity = new Vec2(velocity.x, this.jumpForce);
    this.isGrounded = false;
  }

  private onBeginContact(
    selfCollider: Collider2D,
    otherCollider: Collider2D,
```

```

TypeScript
otherCollider: Collider2D,
contact: IPhysics2DContact | null
) {
  if {
    otherCollider.node.name === 'Ground' ||
    otherCollider.node.name === 'LeftWall' ||
    otherCollider.node.name === 'RightWall'
  } {
    this.isGrounded = true;
  }
}

private onEndContact(
  selfCollider: Collider2D,
  otherCollider: Collider2D,
  contact: IPhysics2DContact | null
) {
  if {
    otherCollider.node.name === 'Ground' ||
    otherCollider.node.name === 'LeftWall' ||
    otherCollider.node.name === 'RightWall'
  } {
    this.isGrounded = false;
  }
}
}
}

```

Attach this script to `Player`.

7. Create QuestionBlock

Under `World`, create:

QuestionBlock

Add:

Sprite
RigidBody2D
BoxCollider2D
QuestionBlock script

Set:

RigidBody2D Type: Static
Position X: 0
Position Y: -80
Width: 50
Height: 50

Put it above the ground so the player can jump and hit it from below.

8. Create Mushroom prefab

Create a new node under `World`:

Mushroom

Add:

Sprite
RigidBody2D
BoxCollider2D
Mushroom script

Set:

RigidBody2D Type: Static
BoxCollider2D Sensor: true

Then drag `Mushroom` from the hierarchy into your `assets/prefabs` folder to make it a prefab.

After making the prefab, you can delete the Mushroom node from the scene.

9. Mushroom code

Create:

assets/scripts/Mushroom.ts

Paste:

```

TypeScript
import {
  _decorator,
  Component,
  Collider2D,
  Contact2DType,
  IPhysics2DContact,
  Vec3
}

```

```

TypeScript
// ...

const { ccclass } = _decorator;

@ccclass('Mushroom')
export class Mushroom extends Component {
    start() {
        const collider = this.getComponent(Collider2D);

        if (collider) {
            collider.on(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
        }

        private onBeginContact(
            selfCollider: Collider2D,
            otherCollider: Collider2D,
            contact: IPhysics2DContact | null
        ) {
            if (otherCollider.node.name !== 'Player') return;

            otherCollider.node.setScale(new Vec3(1.4, 1.4, 1));
            this.node.destroy();
        }
    }
}

```

Attach it to the Mushroom prefab.

10. QuestionBlock code

Create:

assets/scripts/QuestionBlock.ts

Paste:

```

TypeScript
import {
    _decorator,
    Component,
    Collider2D,
    Contact2DType,
    IPhysics2DContact,
    Prefab,
    instantiate,
    Sprite,
    SpriteFrame
} from 'cc';

const { ccclass, property } = _decorator;

@ccclass('QuestionBlock')
export class QuestionBlock extends Component {
    @property(Prefab)
    mushroomPrefab: Prefab | null = null;

    @property(SpriteFrame)
    usedBlockSprite: SpriteFrame | null = null;

    private used: boolean = false;

    start() {
        const collider = this.getComponent(Collider2D);

        if (collider) {
            collider.on(Contact2DType.BEGIN_CONTACT, this.onBeginContact, this);
        }

        private onBeginContact(
            selfCollider: Collider2D,
            otherCollider: Collider2D,
            contact: IPhysics2DContact | null
        ) {
            if (this.used) return;
            if (otherCollider.node.name !== 'Player') return;

            const playerY = otherCollider.node.worldPosition.y;
            const blockY = this.node.worldPosition.y;

            // Player must hit the block from below
            if (playerY < blockY) {
                this.activateBlock();
            }
        }

        private activateBlock() {
            this.used = true;

            const sprite = this.getComponent(Sprite);
            if (sprite && this.usedBlockSprite) {
                sprite.spriteFrame = this.usedBlockSprite;
            }

            if (this.mushroomPrefab) {

```

```

        const mushroom = this.gameObject.GetComponent<Mushroom>();
        this.node.parent?.addChild(mushroom);

        const spawnPos = this.node.worldPosition.clone();
        spawnPos.y += 55;

        mushroom.setWorldPosition(spawnPos);
    }
}

```

Attach it to `QuestionBlock`.

Then in the Inspector:

QuestionBlock script
 — Mushroom Prefab: drag Mushroom prefab here
 — Used Block Sprite: optional

11. Enable physics

Go to:

Project Settings

Make sure 2D physics is enabled. Cocos Creator 3.x uses 2D physics components such as `RigidBody2D`, `BoxCollider2D`, and contact events for collision behavior. The official Cocos docs also note that Creator 3.x and 2.x APIs are not fully compatible, so use the correct docs for your version. [Cocos Docs](#)

Set gravity around:

Gravity X: 0
 Gravity Y: -640

12. Test checklist

Press Preview.

You should be able to test:

- ☐ Scene is no longer black
- ☐ Player falls onto ground
- ☐ Player can move with A/D or arrow keys
- ☐ Player can jump with Space/W/Up
- ☐ Player cannot pass through static ground/walls
- ☐ Player can hit QuestionBlock from below
- ☐ Mushroom appears above block
- ☐ Player touches mushroom and becomes bigger

For your README later, write:

```

<> Markdown

### Level Design

The game scene includes static walls and platforms using Static RigidBody2D and BoxCollider2D. The qu

```

📄 👍 🗨️ 📁 🔄 ... 🌐 Sources

Prompt: do i need a gamemanager?

Answer:

Basic Cocos 2.x GameManager.js

Use this instead:

```
// JavaScript
cc.Class({
    extends: cc.Component,

    properties: {
        player: cc.Node,
        lifeLabel: cc.Label,
        scoreLabel: cc.Label,
        timerLabel: cc.Label,

        maxTime: 120,
        fallLimitY: -400
    },

    onLoad() {
        this.life = 3;
        this.score = 0;
        this.timer = this.maxTime;

        if (this.player) {
            this.startPosition = this.player.position.clone();
        }

        this.updateUI();
    },

    update(dt) {
        this.timer -= dt;

        if (this.timer <= 0) {
            this.timer = 0;
            this.gameOver();
        }

        if (this.player && this.player.y < this.fallLimitY) {
            this.playerDie();
        }

        this.updateUI();
    },

    addScore(amount) {
        this.score += amount;
        this.updateUI();
    },

    playerDie() {
        this.life -= 1;

        if (this.life <= 0) {
            this.gameOver();
            return;
        }

        this.respawnPlayer();
        this.updateUI();
    },

    respawnPlayer() {
        if (!this.player) return;

        this.player.setPosition(this.startPosition);
    },

    gameOver() {
        cc.director.loadScene("GameOver");
    },

    updateUI() {
        if (this.lifeLabel) {
            this.lifeLabel.string = "Life: " + this.life;
        }

        if (this.scoreLabel) {
            this.scoreLabel.string = "Score: " + this.score;
        }

        if (this.timerLabel) {
            this.timerLabel.string = "Time: " + Math.ceil(this.timer);
        }
    }
});
```

Basic Cocos 2.x PlayerController.js

```
JavaScript
cc.Class({
    extends: cc.Component,

    properties: {
        moveSpeed: 250,
        jumpForce: 450
    },

    onLoad() {
        this.moveDir = 0;
        this.isGrounded = false;

        this.rigidBody = this.getComponent(cc.RigidBody);

        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
    },

    onDestroy() {
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
    },

    update(dt) {
        if (!this.rigidBody) return;

        let velocity = this.rigidBody.linearVelocity;
        velocity.x = this.moveDir * this.moveSpeed;
        this.rigidBody.linearVelocity = velocity;
    },

    onKeyDown(event) {
        if (event.keyCode === cc.macro.KEY.a || event.keyCode === cc.macro.KEY.left) {
            this.moveDir = -1;
        }

        if (event.keyCode === cc.macro.KEY.d || event.keyCode === cc.macro.KEY.right) {
            this.moveDir = 1;
        }

        if (
            (event.keyCode === cc.macro.KEY.space ||
             event.keyCode === cc.macro.KEY.w ||
             event.keyCode === cc.macro.KEY.up) &&
            this.isGrounded
        ) {
            this.jump();
        }
    },

    onKeyUp(event) {
        if (
            event.keyCode === cc.macro.KEY.a ||
            event.keyCode === cc.macro.KEY.left ||
            event.keyCode === cc.macro.KEY.d ||
            event.keyCode === cc.macro.KEY.right
        ) {
            this.moveDir = 0;
        }
    },

    jump() {
        if (!this.rigidBody) return;

        let velocity = this.rigidBody.linearVelocity;
        velocity.y = this.jumpForce;
        this.rigidBody.linearVelocity = velocity;

        this.isGrounded = false;
    },

    onBeginContact(contact, selfCollider, otherCollider) {
        if (
            otherCollider.node.name === "Ground" ||
            otherCollider.node.name === "LeftWall" ||
            otherCollider.node.name === "RightWall" ||
            otherCollider.node.name === "PlatformWall"
        ) {
            this.isGrounded = true;
        }
    },

    onEndContact(contact, selfCollider, otherCollider) {
        if (
            otherCollider.node.name === "Ground" ||
            otherCollider.node.name === "LeftWall" ||
            otherCollider.node.name === "RightWall" ||
            otherCollider.node.name === "PlatformWall"
        ) {
            this.isGrounded = false;
        }
    }
});
```

Important setup for Cocos 2.x

For `Player`, use:

```
RigidBody  
Collider  
PlayerController.js
```

Not:

```
RigidBody2D  
BoxCollider2D
```

For `Ground`, `LeftWall`, `RightWall`, use:

```
RigidBody  
BoxCollider
```

Set wall/ground body type to:

```
Static
```

Set player body type to:

```
Dynamic
```

Also enable physics in your main scene with a script or project setting. Add this to a scene manager script if needed:

```
// JavaScript  
cc.director.getPhysicsManager().enabled = true;  
cc.director.getPhysicsManager().gravity = cc.v2(0, -640);
```

So yes, the main issue is: **you are using Cocos 2.x**, so don't use `import ... from 'cc'`.

📄 👍 🗨️ ⚙️ 🔍 ... 📁 Sources

Prompt: oh never mind...the figure do turn left and right. now i need the sprite so it can walk.
how to do that
Answer:

6. Update PlayerController.js

Use this version with animation switching:

```
JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    moveSpeed: 250,
    jumpForce: 520
  },

  onLoad() {
    this.moveDir = 0;
    this.currentAnim = "";

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    this.playAnim("player_idle");

    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  onDestroy() {
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  update(dt) {
    if (!this.rigidBody) return;

    let velocity = this.rigidBody.linearVelocity;
    velocity.x = this.moveDir * this.moveSpeed;
    this.rigidBody.linearVelocity = velocity;

    this.flipPlayer();
    this.updateAnimation();
  },

  onKeyDown(event) {
    if (event.keyCode === cc.macro.KEY.a || event.keyCode === cc.macro.KEY.left) {
      this.moveDir = -1;
    }

    if (event.keyCode === cc.macro.KEY.d || event.keyCode === cc.macro.KEY.right) {
      this.moveDir = 1;
    }

    if (
      event.keyCode === cc.macro.KEY.space ||
      event.keyCode === cc.macro.KEY.w ||
      event.keyCode === cc.macro.KEY.up
    ) {
      this.jump();
    }
  },
});
```

```

onKeyUp(event) {
    if (
        event.keyCode === cc.macro.KEY.a ||
        event.keyCode === cc.macro.KEY.left ||
        event.keyCode === cc.macro.KEY.d ||
        event.keyCode === cc.macro.KEY.right
    ) {
        this.moveDir = 0;
    }
},

jump() {
    if (!this.rigidBody) return;

    let velocity = this.rigidBody.linearVelocity;
    velocity.y = this.jumpForce;
    this.rigidBody.linearVelocity = velocity;
},

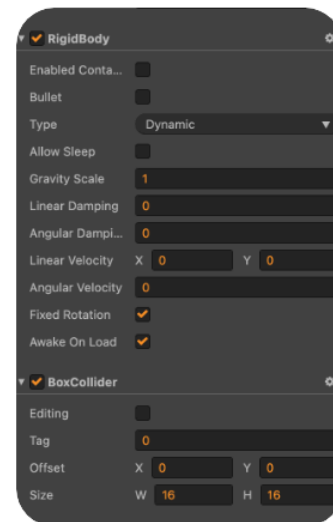
flipPlayer() {
    if (this.moveDir < 0) {
        this.node.scaleX = -Math.abs(this.node.scaleX);
    } else if (this.moveDir > 0) {
        this.node.scaleX = Math.abs(this.node.scaleX);
    }
},

updateAnimation() {
    if (this.moveDir !== 0) {
        this.playAnim("player_walk");
    } else {
        this.playAnim("player_idle");
    }
},

playAnim(animName) {
    if (!this.anim) return;
    if (this.currentAnim === animName) return;

    this.currentAnim = animName;
    this.anim.play(animName);
}
});

```



this is the player. anything wrong?



Thought for 8s >

Your Player setup looks mostly correct.

```
Rigidbody: Dynamic
Gravity Scale: 1
Fixed Rotation: checked
Awake On Load: checked
BoxCollider: exists
PlayerController: attached
```



So the Inspector is okay. The problem is probably your `PlayerController.js`.

Since the player **turns left/right**, keyboard input works. Since the animation plays, the script is running. So your code is probably changing animation and `scaleX`, but not actually changing the player's x position.

Use this version first. It uses direct movement, so it should definitely move:

```
<> JavaScript
cc.Class({
  extends: cc.Component,

  properties: {
    moveSpeed: 250,
    jumpForce: 520
  },

  onLoad() {
    this.moveDir = 0;
    this.currentAnim = "";

    this.rigidBody = this.getComponent(cc.Rigidbody);
    this.anim = this.getComponent(cc.Animation);
```



```

        this.playAnim("player_idle");

        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
    },

    onDestroy() {
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
    },

    update(dt) {
        // Move player left/right
        this.node.x += this.moveDir * this.moveSpeed * dt;

        this.flipPlayer();
        this.updateAnimation();
    },

    onKeyDown(event) {
        if (event.keyCode === cc.macro.KEY.a || event.keyCode === cc.macro.KEY.left) {
            this.moveDir = -1;
        }

        if (event.keyCode === cc.macro.KEY.d || event.keyCode === cc.macro.KEY.right) {
            this.moveDir = 1;
        }

        if (
            event.keyCode === cc.macro.KEY.space ||
            event.keyCode === cc.macro.KEY.w ||
            event.keyCode === cc.macro.KEY.up
        ) {
            this.jump();
        }
    },

    onKeyUp(event) {
        if (
            event.keyCode === cc.macro.KEY.a ||
            event.keyCode === cc.macro.KEY.left ||
            event.keyCode === cc.macro.KEY.d ||
            event.keyCode === cc.macro.KEY.right
        ) {
            this.moveDir = 0;
        }
    },

    jump() {
        if (!this.rigidBody) return;

        let velocity = this.rigidBody.linearVelocity;
        velocity.y = this.jumpForce;
        this.rigidBody.linearVelocity = velocity;
    },

    flipPlayer() {
        if (this.moveDir < 0) {
            this.node.scaleX = -Math.abs(this.node.scaleX);
        } else if (this.moveDir > 0) {
            this.node.scaleX = Math.abs(this.node.scaleX);
        }
    },

```

okay it works.. but nothing happens when i press space. its supposed to jump right



Thought for 8s >

Yes, Space should make the player jump.

Since your left/right works now, replace your `PlayerController.js` with this version. It keeps your working left/right movement, but adds a clearer jump function.

```
JavaScript
cc.Class({
  extends: cc.Component,

  properties: {
    moveSpeed: 120,
    jumpForce: 520
  },

  onLoad() {
    this.moveDir = 0;
    this.currentAnim = "";

    cc.director.getPhysicsManager().enabled = true;
    cc.director.getPhysicsManager().gravity = cc.v2(0, -640);

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    this.playAnim("player_idle");

    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  onDestroy() {
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  update(dt) {
    // Left/right movement
    this.node.x += this.moveDir * this.moveSpeed * dt;

    this.flipPlayer();
    this.updateAnimation();
  },

  onKeyDown(event) {
    if (event.keyCode === cc.macro.KEY.a || event.keyCode === cc.macro.KEY.left) {
      this.moveDir = -1;
    }

    if (event.keyCode === cc.macro.KEY.d || event.keyCode === cc.macro.KEY.right) {
      this.moveDir = 1;
    }
  }
});
```



```

    }
  },

  updateAnimation() {
    if (this.moveDir !== 0) {
      this.playAnim("player_walk");
    } else {
      this.playAnim("player_idle");
    }
  },

  playAnim(animName) {
    if (!this.anim) return;
    if (this.currentAnim === animName) return;

    this.currentAnim = animName;
    this.anim.play(animName);
  }
});

```

Then test:

```

A / Left arrow → move left + walk animation
D / Right arrow → move right + walk animation
No key         → idle animation
Space          → jump

```



One more thing: make sure your player is **not inside the GroundCollider** at the start. If the player starts inside a wall/ground collider, physics can block it. Move the Player a little above the ground.

Your BoxCollider size is only `16 x 16`, which is okay if your Mario sprite is 16x16. If the sprite looks bigger on screen, increase the collider size later. For now, movement should work with this code.



```

        if (
            event.keyCode === cc.macro.KEY.space ||
            event.keyCode === cc.macro.KEY.w ||
            event.keyCode === cc.macro.KEY.up
        ) {
            cc.log("JUMP KEY PRESSED");
            this.jump();
        }
    },

    onKeyUp(event) {
        if (
            event.keyCode === cc.macro.KEY.a ||
            event.keyCode === cc.macro.KEY.left ||
            event.keyCode === cc.macro.KEY.d ||
            event.keyCode === cc.macro.KEY.right
        ) {
            this.moveDir = 0;
        }
    },

    jump() {
        if (!this.rigidBody) {
            cc.log("No RigidBody found");
            return;
        }

        let velocity = this.rigidBody.linearVelocity;
        velocity.y = this.jumpForce;
        this.rigidBody.linearVelocity = velocity;

        cc.log("Jump velocity set to: " + this.jumpForce);
    },

    flipPlayer() {
        if (this.moveDir < 0) {
            this.node.scaleX = -Math.abs(this.node.scaleX);
        } else if (this.moveDir > 0) {
            this.node.scaleX = Math.abs(this.node.scaleX);
        }
    },

    updateAnimation() {
        if (this.moveDir !== 0) {
            this.playAnim("player_walk");
        } else {
            this.playAnim("player_idle");
        }
    },

    playAnim(animName) {
        if (!this.anim) return;
        if (this.currentAnim === animName) return;

        this.currentAnim = animName;
        this.anim.play(animName);
    }
});

```

okay can you help me so the camera follows the player movement? i think it's one of the requirements

Thought for 6s >

Yes, it is one of the requirements. The PDF says the **background and camera should move according to the player's position**. 

1. Create CameraFollow.js

Create this file:

assets/scripts/CameraFollow.js



Paste this:

```
</> JavaScript 
cc.Class({
  extends: cc.Component,

  properties: {
    target: cc.Node,

    followX: true,
    followY: false,

    offsetX: 0,
    offsetY: 0,

    minX: -9999,
    maxX: 9999,
    minY: -9999,
    maxY: 9999,

    smoothSpeed: 5
  },

  update(dt) {
    if (!this.target) return;

    let currentPos = this.node.position;
    let targetX = currentPos.x;
    let targetY = currentPos.y;

    if (this.followX) {
      targetX = this.target.x + this.offsetX;
      targetX = Math.max(this.minX, Math.min(this.maxX, targetX));
    }

    if (this.followY) {
      targetY = this.target.y + this.offsetY;
      targetY = Math.max(this.minY, Math.min(this.maxY, targetY));
    }

    let newX = currentPos.x + (targetX - currentPos.x) * this.smoothSpeed * dt;
    let newY = currentPos.y + (targetY - currentPos.y) * this.smoothSpeed * dt;
```

camera still not moving. also can you make it so the camera is more zoomed in

Thought for 21s >

Yes. Let's fix both things: **camera follow** and **zoom in**.

1. Use this CameraFollow.js

Create/replace:

assets/scripts/CameraFollow.js



Paste this:

```
JavaScript
cc.Class({
  extends: cc.Component,

  properties: {
    target: cc.Node,

    followX: true,
    followY: false,

    offsetX: 120,
    offsetY: 0,

    minX: -9999,
    maxX: 9999,

    followSpeed: 5,

    zoomRatio: 1.8
  },

  onLoad() {
    cc.log("CameraFollow is running");

    this.camera = this.getComponent(cc.Camera);

    if (this.camera) {
      this.camera.zoomRatio = this.zoomRatio;
    } else {
      cc.log("No Camera component found on this node");
    }

    if (!this.target) {
      cc.log("Camera target is not assigned");
    }
  },

  lateUpdate(dt) {
    if (!this.target) return;

    let targetWorldPos = this.target.convertToWorldSpaceAR(cc.v2(0, 0));
```

```
let targetLocalPos = this.node.parent.convertToNodeSpaceAR(targetWorldPos);

let targetX = this.node.x;
let targetY = this.node.y;

if (this.followX) {
    targetX = targetLocalPos.x + this.offsetX;
    targetX = Math.max(this.minX, Math.min(this.maxX, targetX));
}

if (this.followY) {
    targetY = targetLocalPos.y + this.offsetY;
}

this.node.x += (targetX - this.node.x) * this.followSpeed * dt;
this.node.y += (targetY - this.node.y) * this.followSpeed * dt;
}
});
```

2 Attach it correctly

okay but when i make the zoom bigger, it points to the sky instead of the player

Thought for 7s >

Yes, that happens because **zooming in makes the camera view smaller**, so if the camera center is too high, it shows mostly sky.

For Mario-style games, usually:

Camera follows player X
Camera Y stays fixed



So the camera moves left/right, but does not follow Mario's jump.

Replace `CameraFollow.js` with this

```
</> JavaScript
cc.Class({
  extends: cc.Component,

  properties: {
    target: cc.Node,

    offsetX: 120,

    // lower this if camera shows too much sky
    fixedY: -80,

    minX: -9999,
    maxX: 9999,

    followSpeed: 5,

    zoomRatio: 2
  },

  onLoad() {
    this.camera = this.getComponent(cc.Camera);

    if (this.camera) {
      this.camera.zoomRatio = this.zoomRatio;
    }

    cc.log("CameraFollow running");
  },

  lateUpdate(dt) {
    if (!this.target) return;

    let targetWorldPos = this.target.convertToWorldSpaceAR(cc.v2(0, 0));
    let targetLocalPos = this.node.parent.convertToNodeSpaceAR(targetWorldPos);

    let targetX = targetLocalPos.x + this.offsetX;
    targetX = Math.max(this.minX, Math.min(this.maxX, targetX));

    this.node.x += (targetX - this.node.x) * this.followSpeed * dt;
  }
});
```



```

        let targetX = targetLocalPos.x + this.offsetX;

        // clamp camera position
        targetX = Math.max(minCameraX, Math.min(maxCameraX, targetX));

        this.node.x += (targetX - this.node.x) * this.followSpeed * dt;
        this.node.y = this.fixedY;
    }
});

```

Then in **Main Camera** → **CameraFollow**, set:

```

Target: Player
Zoom Ratio: 2
Level Left X: 0
Level Right X: your map ending x
Fixed Y: adjust vertical view
Offset X: 80 or 120

```

For now, try:

```

Level Left X: 0
Level Right X: 2000
Zoom Ratio: 2
Fixed Y: 0
Offset X: 100

```

If your map is longer, increase:

```

Level Right X

```

Example:

```

Level Right X: 3000

```

Main idea:

```

levelLeftX = where your map starts
levelRightX = where your map ends

```

This prevents the camera from showing black outside the left edge of the map.

no i meant the camera starts at $x = 0$. as you can right now, there's the black part.

Thought for 13s >

Yes, I get it now. The black part appears because the **camera center goes too far left**.

For a camera, $x = 0$ means the **center** of the camera is at 0, so half of the screen is showing the area left of 0. That is why you see black.

So the camera should not start at $x = 0$. It should start at:

half of the visible camera width

Use this `CameraFollow.js` instead:

```
JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    target: cc.Node,

    offsetX: 120,
    fixedY: 0,

    followSpeed: 5,
    zoomRatio: 2,

    // left edge and right edge of your whole map
    levelLeftX: 0,
    levelRightX: 2000
  },

  onLoad() {
    this.camera = this.getComponent(cc.Camera);

    if (this.camera) {
      this.camera.zoomRatio = this.zoomRatio;
    }
  },

  lateUpdate(dt) {
    if (!this.target) return;

    let targetWorldPos = this.target.convertToWorldSpaceAR(cc.v2(0, 0));
    let targetLocalPos = this.node.parent.convertToNodeSpaceAR(targetWorldPos);

    let visibleSize = cc.view.getVisibleSize();

    // how much the camera can see after zoom
    let halfCameraWidth = visibleSize.width / 2 / this.zoomRatio;

    // camera center cannot go past these values
    let minCameraX = this.levelLeftX + halfCameraWidth;
    let maxCameraX = this.levelRightX - halfCameraWidth;
```


Prompt: okay solved that now help me with – Enemies: (15%)

- Enemies should have correct physics properties.
- At least 1 type of enemies.
- Only when player hits on their heads can kill them.

– Question Blocks: (5%)

- At least 1 type of blocks (super mushroom which makes Mario bigger).
- Blocks can interact with player correctly.

Answer:

2. Create EnemyController.js

Create:

assets/scripts/EnemyController.js

Paste this:

```
JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    speed: 50,
    gameManager: cc.Node,
    bounceForce: 350
  },

  onLoad() {
    this.direction = -1;
    this.isDead = false;

    this.rigidBody = this.getComponent(cc.RigidBody);

    if (this.rigidBody) {
      this.rigidBody.enabledContactListener = true;
    }
  },

  update(dt) {
    if (this.isDead) return;
    if (!this.rigidBody) return;

    let velocity = this.rigidBody.linearVelocity;
    velocity.x = this.direction * this.speed;
    this.rigidBody.linearVelocity = velocity;
  },

  onBeginContact(contact, selfCollider, otherCollider) {
    if (this.isDead) return;

    let otherNode = otherCollider.node;
    let otherName = otherNode.name;

    // Player touches enemy
    if (otherName === "Player") {
      this.handlePlayerContact(otherNode);
      return;
    }

    // Enemy touches wall/block/pipe, then turn around
    if (
      otherName.includes("Wall") ||
      otherName.includes("Block") ||
      otherName.includes("Pipe") ||
      otherName.includes("Collider")
    ) {
      // Do not reverse only because of the floor
      if (!otherName.includes("Ground")) {
        this.direction *= -1;
      }
    }
  }
});
```



```

    }
},

handlePlayerContact(playerNode) {
    let playerRigidBody = playerNode.getComponent(cc.RigidBody);

    let playerIsAbove = playerNode.y > this.node.y + 8;
    let playerIsFalling = true;

    if (playerRigidBody) {
        playerIsFalling = playerRigidBody.linearVelocity.y <= 0;
    }

    // Player kills enemy only by landing on its head
    if (playerIsAbove && playerIsFalling) {
        this.killEnemy(playerNode);
    } else {
        this.hurtPlayer();
    }
},

killEnemy(playerNode) {
    this.isDead = true;

    // Bounce player upward after killing enemy
    let playerRigidBody = playerNode.getComponent(cc.RigidBody);
    if (playerRigidBody) {
        playerRigidBody.linearVelocity = cc.v2(
            playerRigidBody.linearVelocity.x,
            this.bounceForce
        );
    }

    // Add score
    if (this.gameManager) {
        let gm = this.gameManager.getComponent("GameManager");
        if (gm) {
            gm.addScore(100);
        }
    }

    this.node.destroy();
},

hurtPlayer() {
    if (!this.gameManager) return;

    let gm = this.gameManager.getComponent("GameManager");
    if (gm) {
        gm.playerDie();
    }
}
});

```

4. Create QuestionBlock.js

Create:

assets/scripts/QuestionBlock.js



Paste this:

JavaScript



```
cc.Class({
  extends: cc.Component,

  properties: {
    mushroomPrefab: cc.Prefab,
    usedBlockSprite: cc.SpriteFrame
  },

  onLoad() {
    this.used = false;

    let rb = this.getComponent(cc.RigidBody);
    if (rb) {
      rb.enabledContactListener = true;
    }
  },

  onBeginContact(contact, selfCollider, otherCollider) {
    if (this.used) return;
    if (otherCollider.node.name !== "Player") return;

    let player = otherCollider.node;
    let playerRigidBody = player.getComponent(cc.RigidBody);

    let playerIsBelow = player.y < this.node.y;
    let playerIsMovingUp = true;

    if (playerRigidBody) {
      playerIsMovingUp = playerRigidBody.linearVelocity.y > 0;
    }

    // Only activate when player hits block from below
    if (playerIsBelow && playerIsMovingUp) {
      this.activateBlock();
    }
  },

  activateBlock() {
    this.used = true;

    // Change to used block sprite, optional
    let sprite = this.getComponent(cc.Sprite);
    if (sprite && this.usedBlockSprite) {
      sprite.spriteFrame = this.usedBlockSprite;
    }

    // Spawn mushroom above the block
    if (this.mushroomPrefab) {
      let mushroom = cc.instantiate(this.mushroomPrefab);

      this.node.parent.addChild(mushroom);
    }
  }
});
```

3. Create `Mushroom.js`

Create:

assets/scripts/Mushroom.js



Paste this:

```
// JavaScript
cc.Class({
  extends: cc.Component,

  properties: {
    growScale: 1.5,
    gameManager: cc.Node
  },

  onLoad() {
    let rb = this.getComponent(cc.RigidBody);
    if (rb) {
      rb.enabledContactListener = true;
    }
  },

  onBeginContact(contact, selfCollider, otherCollider) {
    if (otherCollider.node.name !== "Player") return;

    let player = otherCollider.node;

    // Keep facing direction while making Mario bigger
    if (player.scaleX < 0) {
      player.scaleX = -this.growScale;
    } else {
      player.scaleX = this.growScale;
    }

    player.scaleY = this.growScale;

    if (this.gameManager) {
      let gm = this.gameManager.getComponent("GameManager");
      if (gm) {
        gm.addScore(50);
      }
    }

    this.node.destroy();
  }
});
```



Attach this script to the Mushroom prefab.

Prompt: can you help me with the flower first? i have the open mouth and close mouth flower and i already make the animation clip. how to make it so it comes up and down from the tube like mario game

Answer:

Yes. Make the flower as an **enemy node** with its own script attached to the flower node.

1. Node setup

In your hierarchy:

```
World
├─ Enemies
│   └─ FlowerEnemy
├─ StaticWalls
│   └─ PipeCollider
├─ TileMap
└─ Player
```



Select **FlowerEnemy** and add:

```
Sprite
Animation
RigidBody
Physics Box Collider
FlowerEnemy.js
```



Important settings:

```
RigidBody Type: Kinematic
Gravity Scale: 0
Fixed Rotation: checked
Awake On Load: checked
Enabled Contact Listener: checked
```



Physics Box Collider:

```
Sensor: checked
```



Use **Physics Box Collider**, not normal BoxCollider.

Your pipe should still have its own static collider:

```
PipeCollider
├─ RigidBody: Static
└─ Physics Box Collider
```



2. Put flower in the tube

Place **FlowerEnemy** at the hidden position first, inside the pipe.

Example:

```
Flower hidden position: inside tube
Move Distance: 48 or 64
```



When the script runs, it will move upward from that starting position, wait, then move back down.

assets/scripts/FlowerEnemy.js



Paste this:

</> JavaScript



```
cc.Class({
  extends: cc.Component,

  properties: {
    player: cc.Node,
    gameManager: cc.Node,

    moveDistance: 48,
    moveSpeed: 40,

    waitDownTime: 1.2,
    waitUpTime: 1.0,

    stayHiddenDistance: 80,

    canBeStomped: true,
    bounceForce: 350
  },

  onLoad() {
    this.startY = this.node.y;
    this.hiddenY = this.startY;
    this.shownY = this.startY + this.moveDistance;

    this.state = "waitDown";
    this.timer = this.waitDownTime;
    this.dead = false;

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    if (this.rigidBody) {
      this.rigidBody.enabledContactListener = true;
      this.rigidBody.linearVelocity = cc.v2(0, 0);
    }

    if (this.anim) {
      this.anim.play();
    }
  },

  update(dt) {
    if (this.dead) return;

    if (this.state === "waitDown") {
      this.stopMoving();

      // Mario-style behavior: flower stays down if player is near the pipe
      if (this.player && Math.abs(this.player.x - this.node.x) < this.stayHiddenDistance) {
        this.timer = this.waitDownTime;
        return;
      }
    }

    this.timer -= dt;
  }
});
```



```

        let playerIsAbove = playerNode.y > this.node.y + 10;
        let playerIsFalling = false;

        if (playerRb) {
            playerIsFalling = playerRb.linearVelocity.y <= 0;
        }

        // For assignment requirement: only head stomp kills enemy
        if (this.canBeStomped && playerIsAbove && playerIsFalling) {
            this.killFlower(playerNode);
        } else {
            this.hurtPlayer();
        }
    },

    killFlower(playerNode) {
        this.dead = true;
        this.stopMoving();

        let playerRb = playerNode.getComponent(cc.RigidBody);
        if (playerRb) {
            playerRb.linearVelocity = cc.v2(playerRb.linearVelocity.x, this.bounceForce);
        }

        if (this.gameManager) {
            let gm = this.gameManager.getComponent("GameManager");
            if (gm) {
                gm.addScore(100);
            }
        }

        this.node.destroy();
    },

    hurtPlayer() {
        if (!this.gameManager) return;

        let gm = this.gameManager.getComponent("GameManager");
        if (gm) {
            gm.playerDie();
        }
    }
});

```

```

        if (this.timer <= 0) {
            this.state = "movingUp";
        }
    }

    else if (this.state === "movingUp") {
        this.moveVertical(this.moveSpeed);

        if (this.node.y >= this.shownY) {
            this.node.y = this.shownY;
            this.stopMoving();

            this.state = "waitUp";
            this.timer = this.waitUpTime;
        }
    }

    else if (this.state === "waitUp") {
        this.stopMoving();

        this.timer -= dt;

        if (this.timer <= 0) {
            this.state = "movingDown";
        }
    }

    else if (this.state === "movingDown") {
        this.moveVertical(-this.moveSpeed);

        if (this.node.y <= this.hiddenY) {
            this.node.y = this.hiddenY;
            this.stopMoving();

            this.state = "waitDown";
            this.timer = this.waitDownTime;
        }
    }
},

moveVertical(speed) {
    if (this.rigidBody) {
        this.rigidBody.linearVelocity = cc.v2(0, speed);
    } else {
        this.node.y += speed * cc.director.getDeltaTime();
    }
},

stopMoving() {
    if (this.rigidBody) {
        this.rigidBody.linearVelocity = cc.v2(0, 0);
    }
},

onBeginContact(contact, selfCollider, otherCollider) {
    if (this.dead) return;
    if (otherCollider.node.name !== "Player") return;

    let playerNode = otherCollider.node;
    let playerRb = playerNode.getComponent(cc.RigidBody);

```

let's just circle back. help me with the enemy first? the one that i can kill by jumping on the head

Thought for 22s >

Yes, let's do the normal **Goomba-style enemy** first.

This enemy will work like this:

Mario jumps on enemy head → enemy dies
Mario touches enemy from side → Mario loses 1 life



1. Enemy node setup

Create this in your hierarchy:

```
World
├── Enemies
│   └── Enemy_1
├── Player
└── Game Manager
```



Select **Enemy_1** and add:

Sprite
Animation optional
RigidBody
Physics Box Collider
EnemyController



Set **Enemy_1** RigidBody:

Type: Dynamic
Gravity Scale: 1
Fixed Rotation: checked
Awake On Load: checked
Enabled Contact Listener: checked



Set **Physics Box Collider**:

Sensor: unchecked
Size: match enemy body



2. Full EnemyController.js

Create:

assets/scripts/EnemyController.js



Paste this:

okay but i should die when i come into contact with the flower right

Thought for 8s >

Yes. For the **flower enemy**, Mario should lose life when he touches it.

Since it is like a Mario pipe flower, the simplest rule is:

Player touches flower → player loses life



So for the flower, you can make it **not stompable**.

Change this in `FlowerEnemy.js`

Find this part:

```
<> JavaScript
onBeginContact(contact, selfCollider, otherCollider) {
  if (this.dead) return;
  if (otherCollider.node.name !== "Player") return;

  let playerNode = otherCollider.node;
  let playerRb = playerNode.getComponent(cc.RigidBody);

  let playerIsAbove = playerNode.y > this.node.y + 10;
  let playerIsFalling = false;

  if (playerRb) {
    playerIsFalling = playerRb.linearVelocity.y <= 0;
  }

  if (this.canBeStomped && playerIsAbove && playerIsFalling) {
    this.killFlower(playerNode);
  } else {
    this.hurtPlayer();
  }
},
```



Replace it with this:

```
<> JavaScript
onBeginContact(contact, selfCollider, otherCollider) {
  if (this.dead) return;
  if (otherCollider.node.name !== "Player") return;

  // Touching the flower hurts Mario
  this.hurtPlayer();
},
```



Now the rule is simple:

```

        this.rigidBody.linearVelocity = velocity;
    } else {
        this.node.x += this.direction * this.speed * cc.director.getDeltaTime();
    }

    // Flip sprite direction
    if (this.direction < 0) {
        this.node.scaleX = -Math.abs(this.node.scaleX);
    } else {
        this.node.scaleX = Math.abs(this.node.scaleX);
    }
},

onBeginContact(contact, selfCollider, otherCollider) {
    if (this.dead) return;
    if (!this.player) return;

    if (otherCollider.node === this.player) {
        this.handlePlayerContact();
    }
},

handlePlayerContact() {
    if (!this.player) return;

    let playerRb = this.player.getComponent(cc.RigidBody);

    let playerBox = this.player.getBoundingBoxToWorld();
    let enemyBox = this.node.getBoundingBoxToWorld();

    let playerBottom = playerBox.y;
    let enemyTop = enemyBox.y + enemyBox.height;

    let playerIsAboveEnemy = playerBottom >= enemyTop - this.stompTolerance;

    let playerIsFalling = true;

    if (playerRb) {
        playerIsFalling = playerRb.linearVelocity.y <= 0;
    }

    if (playerIsAboveEnemy && playerIsFalling) {
        this.killEnemy();
    } else {
        this.hurtPlayer();
    }
},

killEnemy() {
    if (this.dead) return;

    this.dead = true;

    cc.log("Enemy killed by head stomp");

    // Bounce Mario upward
    if (this.player) {
        let playerRb = this.player.getComponent(cc.RigidBody);

        if (playerRb) {
            playerRb.linearVelocity = cc.v2(
                playerRb.linearVelocity.x,
                this.bounceForce
            );
        }
    }
}

```

```

    }

    // Add score
    let gm = this.getGameManager();

    if (gm) {
        gm.addScore(100);
    }

    this.node.destroy();
},

hurtPlayer() {
    if (!this.canDamage) return;

    this.canDamage = false;

    cc.log("Enemy hurt player");

    let gm = this.getGameManager();

    if (gm) {
        gm.playerDie();
    } else {
        cc.log("GameManager not found");
    }

    this.scheduleOnce(function () {
        this.canDamage = true;
    }, this.damageCooldown);
},

getGameManager() {
    if (!this.gameManager) {
        cc.log("Game Manager node is not assigned to EnemyController");
        return null;
    }

    let gm = this.gameManager.getComponent("GameManager");

    // Backup: find any component that has playerDie()
    if (!gm) {
        let components = this.gameManager.getComponents(cc.Component);

        for (let i = 0; i < components.length; i++) {
            if (typeof components[i].playerDie === "function") {
                gm = components[i];
                break;
            }
        }
    }

    return gm;
}
});

```

okay now for timer, why isnt it counting down



Thought for 7s >

It's probably because the **TimerLabel** is not connected to your **GameManager**.

Your screenshot shows:

T120



That looks like you manually typed the label text, but GameManager is not updating it yet.

1. Connect TimerLabel to GameManager

Select your **Game Manager** node.

In the Inspector, find your **GameManager** script component.

You should see:

Player
Life Label
Score Label
Timer Label



Drag your actual timer label from the hierarchy into:

Timer Label



Example:

Canvas
└─ HUD
 └─ TimerLabel ← drag this into Timer Label slot



Do the same for life and score too.

2. Change `updateUI()` to this

In your `GameManager.js`, replace your `updateUI()` with:

```
</> JavaScript
updateUI() {
  if (this.lifeLabel) {
    this.lifeLabel.string = "L" + this.life;
  }

  if (this.scoreLabel) {
    this.scoreLabel.string = this.score.toString();
  }

  if (this.timerLabel) {
    this.timerLabel.string = "T" + Math.ceil(this.timer);
  }
}
```



JavaScript



```
cc.Class({
  extends: cc.Component,

  properties: {
    player: cc.Node,
    gameManager: cc.Node,

    speed: 40,

    // Enemy patrol distance from its starting point
    leftDistance: 80,
    rightDistance: 80,

    // How high Mario must be to count as a head stomp
    stompTolerance: 8,

    bounceForce: 350,
    damageCooldown: 1.0
  },

  onLoad() {
    this.startX = this.node.x;
    this.direction = -1;
    this.dead = false;
    this.canDamage = true;

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    if (this.rigidBody) {
      this.rigidBody.enabledContactListener = true;
      this.rigidBody.linearVelocity = cc.v2(0, 0);
    }

    if (this.anim) {
      this.anim.play();
    }

    cc.log("EnemyController is running");
  },

  update(dt) {
    if (this.dead) return;

    this.updateMovement();
  },

  updateMovement() {
    // Turn around based on patrol range
    if (this.node.x <= this.startX - this.leftDistance) {
      this.direction = 1;
    }

    if (this.node.x >= this.startX + this.rightDistance) {
      this.direction = -1;
    }

    // Move using physics velocity
    if (this.rigidBody) {
      let velocity = this.rigidBody.linearVelocity;
```


okay now it works. but after i jump and kill the enemy, it dissapears right. i want it to be like a real mario game. the turtle turns into shell

Thought for 11s >

Yes. Instead of destroying the turtle after Mario stomps it, change it into a **shell state**.

The logic will be:

```
Mario jumps on turtle head → turtle becomes shell
Shell stays on ground
Mario bounces upward
Score increases
```

Replace your full `EnemyController.js`

```
// JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    player: cc.Node,
    gameManager: cc.Node,

    speed: 40,

    stompTolerance: 10,
    bounceForce: 350,

    damageCooldown: 1.0,

    // Drag your shell sprite frame here
    shellSpriteFrame: cc.SpriteFrame
  },

  onLoad() {
    cc.log("Turtle EnemyController running");

    this.direction = -1;
    this.state = "walking";
    this.canDamage = true;

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);
    this.sprite = this.getComponent(cc.Sprite);

    if (this.rigidBody) {
      this.rigidBody.enabledContactListener = true;
      this.rigidBody.fixedRotation = true;
    }

    if (this.anim) {
      this.anim.play();
    }
  },

  update(dt) {
```

```

    if (this.state === "walking") {
        this.walk();
    }

    else if (this.state === "shell") {
        this.stopMoving();
    }
},

walk() {
    if (!this.rigidBody) return;

    let velocity = this.rigidBody.linearVelocity;

    this.rigidBody.linearVelocity = cc.v2(
        this.direction * this.speed,
        velocity.y
    );

    // Use this version because your turtle sprite was reversed before
    if (this.direction < 0) {
        this.node.scaleX = Math.abs(this.node.scaleX);
    } else {
        this.node.scaleX = -Math.abs(this.node.scaleX);
    }
},

stopMoving() {
    if (!this.rigidBody) return;

    this.rigidBody.linearVelocity = cc.v2(0, this.rigidBody.linearVelocity.y);
},

onBeginContact(contact, selfCollider, otherCollider) {
    let otherNode = otherCollider.node;

    if (otherNode === this.player) {
        this.handlePlayerContact();
        return;
    }

    if (this.state === "walking" && this.isTurnObject(otherNode)) {
        this.turnAround();
    }
},

handlePlayerContact() {
    if (!this.player) return;

    let playerRb = this.player.getComponent(cc.RigidBody);

    let playerBox = this.player.getBoundingBoxToWorld();
    let enemyBox = this.node.getBoundingBoxToWorld();

    let playerBottom = playerBox.y;
    let enemyTop = enemyBox.y + enemyBox.height;

    let playerIsAboveEnemy = playerBottom >= enemyTop - this.stompTolerance;

    let playerIsFalling = true;

    if (playerRb) {
        playerIsFalling = playerRb.linearVelocity.y <= 0;
    }
}

```

```

    if (this.state === "walking") {
        if (playerIsAboveEnemy && playerIsFalling) {
            this.becomeShell();
        } else {
            this.hurtPlayer();
        }
    }

    },

    else if (this.state === "shell") {
        // Shell does not hurt Mario for now
        // Mario can stand/touch it safely
    }
},

becomeShell() {
    if (this.state === "shell") return;

    cc.log("Turtle became shell");

    this.state = "shell";

    // Stop walking animation
    if (this.anim) {
        this.anim.stop();
    }

    // Change sprite to shell
    if (this.sprite && this.shellSpriteFrame) {
        this.sprite.spriteFrame = this.shellSpriteFrame;
    }

    // Stop movement
    if (this.rigidBody) {
        this.rigidBody.linearVelocity = cc.v2(0, 0);
    }

    // Make it slightly shorter like a shell
    this.node.scaleY = 0.7;

    // Bounce Mario upward
    if (this.player) {
        let playerRb = this.player.getComponent(cc.RigidBody);

        if (playerRb) {
            playerRb.linearVelocity = cc.v2(
                playerRb.linearVelocity.x,
                this.bounceForce
            );
        }
    }

    // Add score
    let gm = this.getGameManager();

    if (gm) {
        gm.addScore(100);
    }
},

isTurnObject(node) {
    if (!node) return false;

```

the mushroom just spawns and stay there right. what if i want the mushroom to spawn and fall to the ground like real mario

Thought for 9s >

Yes, for real Mario behavior, the mushroom should:

come out of the question block
fall down because of gravity
move sideways
turn around when it hits walls/pipes/blocks
get collected when Mario touches it



So now we need to make the **Mushroom** prefab use physics.

1. Set up the Mushroom prefab

Open your **Mushroom** prefab.

Add these components to the root **Mushroom** node:

Sprite
RigidBody
Physics Box Collider
Mushroom.js



Set **RigidBody** :

Type: Dynamic
Fixed Rotation: checked
Enabled Contact Listener: checked
Gravity Scale: 1



Set **Physics Box Collider** :

Sensor: unchecked
Width: 16
Height: 16



Keep:

Group: default



2. Replace full **Mushroom.js**

</> JavaScript



```
cc.Class({
    extends: cc.Component,

    properties: {
        player: cc.Node,
        gameManager: cc.Node,

        emergeDistance: 24,
        emergeSpeed: 35,

        moveSpeed: 45,

        growScale: 1.5,
        scoreAmount: 50,

        collectDelay: 0.3
    },

    onLoad() {
        this.startY = this.node.y;
        this.targetY = this.startY + this.emergeDistance;

        this.state = "emerging";
        this.canCollect = false;
        this.collected = false;

        this.direction = 1;

        this.rigidBody = this.getComponent(cc.RigidBody);

        if (this.rigidBody) {
            this.rigidBody.enabledContactListener = true;
            this.rigidBody.fixedRotation = true;

            // While emerging, do not fall yet
            this.rigidBody.type = cc.RigidBodyType.Kinematic;
            this.rigidBody.linearVelocity = cc.v2(0, 0);
        }

        this.node.active = true;
        this.node.opacity = 255;
        this.node.zIndex = 999;
        this.node.setSiblingIndex(999);

        cc.log("Mushroom spawned");
    },

    update(dt) {
        if (this.collected) return;

        if (this.state === "emerging") {
            this.emerge(dt);
        }

        else if (this.state === "active") {
            this.moveMushroom();
        }
    }
});
```



```

        this.checkPlayerCollect();
    }
},

emerge(dt) {
    this.node.y += this.emergeSpeed * dt;

    if (this.node.y >= this.targetY) {
        this.node.y = this.targetY;
        this.state = "active";

        cc.log("Mushroom finished emerging");

        if (this.rigidBody) {
            // Now gravity works
            this.rigidBody.type = cc.RigidBodyType.Dynamic;
            this.rigidBody.linearVelocity = cc.v2(this.direction * this.moveSpeed, 0);
        }

        this.scheduleOnce(function () {
            this.canCollect = true;
            cc.log("Mushroom can now be collected");
        }, this.collectDelay);
    }
},

moveMushroom() {
    if (!this.rigidBody) return;

    let velocity = this.rigidBody.linearVelocity;

    this.rigidBody.linearVelocity = cc.v2(
        this.direction * this.moveSpeed,
        velocity.y
    );
},

onBeginContact(contact, selfCollider, otherCollider) {
    if (this.collected) return;

    let otherNode = otherCollider.node;

    if (otherNode === this.player || otherNode.name.toLowerCase().includes("player")) {
        if (this.canCollect) {
            this.collectMushroom();
        }
        return;
    }

    if (this.isTurnObject(otherNode)) {
        this.turnAround();
    }
},

isTurnObject(node) {
    if (!node) return false;

    let name = node.name.toLowerCase();

```



```

        if (playerBox.intersects(mushroomBox)) {
            this.collectMushroom();
        }
    },

    collectMushroom() {
        if (this.collected) return;

        this.collected = true;

        cc.log("Mario collected mushroom");

        if (this.rigidBody) {
            this.rigidBody.linearVelocity = cc.v2(0, 0);
        }

        this.makePlayerBigger();
        this.addScore();

        this.node.destroy();
    },

    makePlayerBigger() {
        if (!this.player) return;

        let direction = 1;

        if (this.player.scaleX < 0) {
            direction = -1;
        }

        this.player.scaleX = direction * this.growScale;
        this.player.scaleY = this.growScale;
    },

    addScore() {
        if (!this.gameManager) return;

        let gm = this.gameManager.getComponent("GameManager");

        if (!gm) {
            let components = this.gameManager.getComponents(cc.Component);

            for (let i = 0; i < components.length; i++) {
                if (typeof components[i].addScore === "function") {
                    gm = components[i];
                    break;
                }
            }
        }

        if (gm) {
            gm.addScore(this.scoreAmount);
        }
    }
});

```



okay. let's leave this be for now. need help with life. if mario is big (after getting a mushroom), then if it hits make contact with an enemy, it'll become small instead of dying. if mario is small when making contact with the enemy --> die

Thought for 7s >

Yes, do this in the **GameManager**, not inside every enemy.

The logic should be:

```
Enemy touches Mario
→ GameManager checks Mario size
→ If Mario is big: make Mario small
→ If Mario is small: lose life / respawn
```

1. Update `GameManager.js`

In `onLoad()`, add these:

```
</> JavaScript
this.isBigMario = false;
this.canTakeDamage = true;
this.damageCooldown = 1.0;
```

So your `onLoad()` should include something like:

```
</> JavaScript
onLoad() {
  let physicsManager = cc.director.getPhysicsManager();
  physicsManager.enabled = true;
  physicsManager.gravity = cc.v2(0, -640);
  physicsManager.debugDrawFlags = 0;

  this.life = 3;
  this.score = 0;
  this.timer = this.maxTime;

  this.isBigMario = false;
  this.canTakeDamage = true;
  this.damageCooldown = 1.0;

  if (this.player) {
    this.startPosition = this.player.position.clone();
    this.setMarioSmall();
  }

  this.updateUI();
}
```


2. Add these methods inside `GameManager.js`

Paste these inside the `cc.Class`, under your existing methods:

```
JavaScript
powerUpMario() {
    if (!this.player) return;

    this.isBigMario = true;

    let direction = 1;

    if (this.player.scaleX < 0) {
        direction = -1;
    }

    this.player.scaleX = direction * 1.5;
    this.player.scaleY = 1.5;

    cc.log("Mario became big");
},

setMarioSmall() {
    if (!this.player) return;

    this.isBigMario = false;

    let direction = 1;

    if (this.player.scaleX < 0) {
        direction = -1;
    }

    this.player.scaleX = direction * 1;
    this.player.scaleY = 1;

    cc.log("Mario became small");
},

hurtMario() {
    if (!this.canTakeDamage) return;

    this.canTakeDamage = false;

    if (this.isBigMario) {
        cc.log("Big Mario got hit, becoming small");

        this.setMarioSmall();

        this.scheduleOnce(function () {
            this.canTakeDamage = true;
        }, this.damageCooldown);

        return;
    }
}
```

```

    }

    cc.log("Small Mario got hit, losing life");

    this.playerDie();

    this.scheduleOnce(function () {
        this.canTakeDamage = true;
    }, this.damageCooldown);
},

```

3. Change Mushroom collection

In your `Mushroom.js`, find where you currently call:

```

</> JavaScript
this.makePlayerBigger();

```

Replace it with:

```

</> JavaScript
this.powerUpMario();

```

Then add this method inside `Mushroom.js`:

```

</> JavaScript
powerUpMario() {
    if (!this.gameManager) return;

    let gm = this.gameManager.getComponent("GameManager");

    if (!gm) {
        let components = this.gameManager.getComponents(cc.Component);

        for (let i = 0; i < components.length; i++) {
            if (typeof components[i].powerUpMario === "function") {
                gm = components[i];
                break;
            }
        }
    }

    if (gm) {
        gm.powerUpMario();
    }
},

```

Now the mushroom tells `GameManager`:



4. Change enemy damage

In your enemy script, find this part:

```
</> JavaScript  
gm.playerDie();
```

Replace it with:

```
</> JavaScript  
gm.hurtMario();
```

So your enemy damage part should become:

```
</> JavaScript  
let gm = this.getGameManager();  
  
if (gm) {  
  gm.hurtMario();  
} else {  
  cc.log("GameManager not found");  
}
```

Do the same thing in your `FlowerEnemy.js` too.

Result

Now the behavior becomes:

```
Mario small + touches enemy  
→ life -1  
→ respawn  
  
Mario big + touches enemy  
→ becomes small  
→ no life lost  
  
Mario small again + touches enemy  
→ life -1  
→ respawn
```



This is the safer structure because all Mario size/life logic stays inside `GameManager.js`.

it doesnt work

Thought for 8s >

Most likely problem: Mario becomes visually big, but `GameManager` still thinks he is small.

So use the player's actual scale to check big/small instead of only using `isBigMario`.

1. In `GameManager.js`, replace these 3 functions

Replace your current `powerUpMario()`, `setMarioSmall()`, and `hurtMario()` with this:

```
</> JavaScript
powerUpMario() {
  if (!this.player) return;

  let direction = 1;

  if (this.player.scaleX < 0) {
    direction = -1;
  }

  this.player.scaleX = direction * this.bigMarioScale;
  this.player.scaleY = this.bigMarioScale;

  this.isBigMario = true;

  cc.log("Mario became big");
},

setMarioSmall() {
  if (!this.player) return;

  let direction = 1;

  if (this.player.scaleX < 0) {
    direction = -1;
  }

  this.player.scaleX = direction * this.smallMarioScale;
  this.player.scaleY = this.smallMarioScale;

  this.isBigMario = false;

  cc.log("Mario became small");
},

hurtMario() {
  if (!this.canTakeDamage) return;

  this.canTakeDamage = false;
```

```

let marioIsBig = false;

if (this.player && Math.abs(this.player.scaleY) > 1.1) {
    marioIsBig = true;
}

if (marioIsBig) {
    cc.log("Big Mario got hit, becoming small");

    this.setMarioSmall();

    this.scheduleOnce(function () {
        this.canTakeDamage = true;
    }, this.damageCooldown);

    return;
}

cc.log("Small Mario got hit, losing life");

this.playerDie();

this.scheduleOnce(function () {
    this.canTakeDamage = true;
}, this.damageCooldown);
},

```

2. In every enemy script, make sure it calls this

Find this:

```

</> JavaScript
gm.playerDie();

```

Replace it with:

```

</> JavaScript
gm.hurtMario();

```

So enemy damage should be:

```

</> JavaScript
let gm = this.getGameManager();

if (gm) {
    gm.hurtMario();
} else {
    cc.log("GameManager not found");
}

```



Do this for:

DO THIS FOR:

```
EnemyController.js  
FlowerEnemy.js
```



3. In `Mushroom.js`, make sure collection calls `GameManager`

Inside `collectMushroom()`, it should use:

```
</> JavaScript  
  
this.powerUpMario();
```



not:

```
</> JavaScript  
  
this.makePlayerBigger();
```



So it should look like this:

```
</> JavaScript  
  
collectMushroom() {  
    if (this.collected) return;  
  
    this.collected = true;  
  
    cc.log("Mario collected mushroom");  
  
    this.powerUpMario();  
    this.addScore();  
  
    this.node.destroy();  
},
```



And your `Mushroom.js` needs this function:

```
</> JavaScript  
  
powerUpMario() {  
    if (!this.gameManager) return;  
  
    let gm = this.gameManager.getComponent("GameManager");  
  
    if (!gm) {  
        let components = this.gameManager.getComponents(cc.Component);  
  
        for (let i = 0; i < components.length; i++) {  
            if (typeof components[i].powerUpMario == "function") {  
                gm = components[i];  
                break;  
            }  
        }  
    }  
}
```



- Sound Effects (10%)
- At least one BGM (2%)
- Player Jump & die sound effects (3%)
- Additional sound effects (each for 1%, up to 5%)
- All sound effects can't stop BGM help please

Thought for 8s >

Yes. For Cocos 2.x, use this structure:

```
BGM → cc.audioEngine.playMusic()  
Sound effects → cc.audioEngine.playEffect()
```



That way, sound effects will **not stop the BGM**.

1. Create SoundManager.js

Create a new script:

```
assets/scripts/SoundManager.js
```



Paste this full code:

```
</> JavaScript  
cc.Class({  
  extends: cc.Component,  
  
  properties: {  
    bgm: cc.AudioClip,  
  
    jumpSfx: cc.AudioClip,  
    dieSfx: cc.AudioClip,  
    hurtSfx: cc.AudioClip,  
  
    mushroomSfx: cc.AudioClip,  
    questionBlockSfx: cc.AudioClip,  
    enemyStompSfx: cc.AudioClip,  
    collectSfx: cc.AudioClip,  
  
    bgmVolume: 0.5,  
    sfxVolume: 1.0  
  },  
  
  onLoad() {  
    SoundManager.instance = this;  
  
    cc.log("SoundManager is running");
```



```
cc.log('SoundManager is running');

    this.playBGM();
},

playBGM() {
    if (!this.bgm) {
        cc.log("BGM is not assigned");
        return;
    }

    cc.audioEngine.setMusicVolume(this.bgmVolume);
    cc.audioEngine.playMusic(this.bgm, true);

    cc.log("BGM started");
},

playSFX(clip) {
    if (!clip) return;

    cc.audioEngine.setEffectsVolume(this.sfxVolume);
    cc.audioEngine.playEffect(clip, false);
},

playJump() {
    this.playSFX(this.jumpSfx);
},

playDie() {
    this.playSFX(this.dieSfx);
},

playHurt() {
    this.playSFX(this.hurtSfx);
},

playMushroom() {
    this.playSFX(this.mushroomSfx);
},

playQuestionBlock() {
    this.playSFX(this.questionBlockSfx);
},

playEnemyStomp() {
    this.playSFX(this.enemyStompSfx);
},

playCollect() {
    this.playSFX(this.collectSfx);
}
});
```



```

    // Do not turn around when touching ground
    if (name.includes("ground")) return false;

    return (
        name.includes("pipe") ||
        name.includes("block") ||
        name.includes("wall") ||
        name.includes("tube")
    );
},

turnAround() {
    this.direction *= -1;

    if (this.rigidBody) {
        this.rigidBody.linearVelocity = cc.v2(
            this.direction * this.speed,
            this.rigidBody.linearVelocity.y
        );
    }

    // Small push so it does not get stuck
    this.node.x += this.direction * 2;

    cc.log("Turtle turned around");
},

hurtPlayer() {
    if (!this.canDamage) return;

    this.canDamage = false;

    cc.log("Turtle hurt player");

    let gm = this.getGameManager();

    if (gm) {
        gm.playerDie();
    } else {
        cc.log("GameManager not found");
    }

    this.scheduleOnce(function () {
        this.canDamage = true;
    }, this.damageCooldown);
},

getGameManager() {
    if (!this.gameManager) {
        cc.log("Game Manager node is not assigned");
        return null;
    }

    let gm = this.gameManager.getComponent("GameManager");

    if (!gm) {
        let components = this.gameManager.getComponents(cc.Component);

        for (let i = 0; i < components.length; i++) {
            if (typeof components[i].playerDie === "function") {
                gm = components[i];
            }
        }
    }
}

```

3. Add jump sound in `PlayerController.js`

Find your `jump()` function.

Use this version:

```
JavaScript

jump() {
  if (!this.rigidBody) return;

  let velocity = this.rigidBody.linearVelocity;
  velocity.y = this.jumpForce;
  this.rigidBody.linearVelocity = velocity;

  if (SoundManager.instance) {
    SoundManager.instance.playJump();
  }
}
```

Now Mario jump gives sound.

4. Add die / hurt sound in `GameManager.js`

In `hurtMario()`, add hurt sound when big Mario becomes small.

Use this part:

```
JavaScript

if (marioIsBig) {
  cc.log("Big Mario got hit, becoming small");

  if (SoundManager.instance) {
    SoundManager.instance.playHurt();
  }

  this.setMarioSmall();

  this.scheduleOnce(function () {
    this.canTakeDamage = true;
  }, this.damageCooldown);

  return;
}
```

In `playerDie()`, add die sound near the top:

```
JavaScript

playerDie() {
  cc.log("playerDie called");
```

5. Add mushroom sound in `Mushroom.js`

In `collectMushroom()`, add this:

```
JavaScript
collectMushroom() {
    if (this.collected) return;

    this.collected = true;

    cc.log("Mario collected mushroom");

    if (SoundManager.instance) {
        SoundManager.instance.playMushroom();
    }

    this.powerUpMario();
    this.addScore();

    this.node.destroy();
},
```

6. Add question block sound in `QuestionBox.js`

In `activateBlock()`, add this:

```
JavaScript
activateBlock() {
    if (this.used) return;

    this.used = true;

    cc.log("QUESTION BOX ACTIVATED");

    if (SoundManager.instance) {
        SoundManager.instance.playQuestionBlock();
    }

    this.spawnMushroom();
}
```

If your version still has `bumpBlock()`, keep it:

```
JavaScript
this.bumpBlock();
this.spawnMushroom();
```



okay now help me add the sounds

Thought for 9s >

Yes. Let's add the sounds safely.

1. Replace full SoundManager.js

Use this version instead. It uses your audio names.

```
JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    // BGM
    bgm_1: cc.AudioClip,
    bgm_2: cc.AudioClip,
    bgm_3: cc.AudioClip,

    // SFX
    coin: cc.AudioClip,
    gameOver: cc.AudioClip,
    gameOver2: cc.AudioClip,
    jump: cc.AudioClip,
    kick: cc.AudioClip,
    levelClear: cc.AudioClip,
    loseOneLife: cc.AudioClip,
    powerDown: cc.AudioClip,
    powerUp: cc.AudioClip,
    powerUpAppear: cc.AudioClip,
    reserve: cc.AudioClip,
    stomp: cc.AudioClip,

    bgmVolume: 0.5,
    sfxVolume: 1.0
  },

  onLoad() {
    window.SoundManager = this;

    cc.log("SoundManager is running");

    this.playBGM1();
  },

  playBGM1() {
    if (!this.bgm_1) return;

    cc.audioEngine.setMusicVolume(this.bgmVolume);
    cc.audioEngine.playMusic(this.bgm_1, true);
  },
});
```

```
playBGM2() {
    if (!this.bgm_2) return;

    cc.audioEngine.setMusicVolume(this.bgmVolume);
    cc.audioEngine.playMusic(this.bgm_2, true);
},

playBGM3() {
    if (!this.bgm_3) return;

    cc.audioEngine.setMusicVolume(this.bgmVolume);
    cc.audioEngine.playMusic(this.bgm_3, true);
},

playSFX(clip) {
    if (!clip) return;

    cc.audioEngine.setEffectsVolume(this.sfxVolume);
    cc.audioEngine.playEffect(clip, false);
},

playCoin() {
    this.playSFX(this.coin);
},

playGameOver() {
    this.playSFX(this.gameOver);
},

playGameOver2() {
    this.playSFX(this.gameOver2);
},

playJump() {
    this.playSFX(this.jump);
},

playKick() {
    this.playSFX(this.kick);
},

playLevelClear() {
    this.playSFX(this.levelClear);
},

playLoseOneLife() {
    this.playSFX(this.loseOneLife);
},

playPowerDown() {
    this.playSFX(this.powerDown);
},

playPowerUp() {
    this.playSFX(this.powerUp);
},

playPowerUpAppear() {
```



Prompt: need help for the question block. this one spawns mushroom right. i want some question block to spawn coins

Answer:

Yes. Make **one** `QuestionBox.js` **script** that can choose the reward:

```
rewardType = "mushroom" → spawns mushroom  
rewardType = "coin" → spawns coin + adds score
```



Then each question block can use the same script, but different Inspector settings.

Replace your full `QuestionBox.js` with this:

```
<> JavaScript  
  
cc.Class({  
  extends: cc.Component,  
  
  properties: {  
    player: cc.Node,  
    gameManager: cc.Node,  
  
    // Type "mushroom" or "coin" in Inspector  
    rewardType: "mushroom",  
  
    mushroomPrefab: cc.Prefab,  
    coinPrefab: cc.Prefab,  
  
    coinScore: 100,  
  
    spawnOffsetY: 18,  
    bumpHeight: 6,  
    bumpTime: 0.08,  
  
    coinMoveUpDistance: 32,  
    coinMoveTime: 0.35,  
  
    idleAnimName: "question_idle",  
    usedSpriteFrame: cc.SpriteFrame  
  },  
  
  onLoad() {  
    this.used = false;  
    this.originalY = this.node.y;  
  
    this.sprite = this.getComponent(cc.Sprite);  
    this.anim = this.getComponent(cc.Animation);  
  
    if (this.anim) {  
      this.anim.play(this.idleAnimName);  
    }  
  
    cc.log("QuestionBox running on: " + this.node.name);  
  
    let rb = this.getComponent(cc.RigidBody);  
  
    if (rb) {  
      rb.enabledContactListener = true;  
    }  
  },  
},
```



```

onBeginContact(contact, selfCollider, otherCollider) {
    let otherNode = otherCollider.node;

    let isPlayer =
        otherNode === this.player ||
        otherNode.name.toLowerCase().includes("player") ||
        otherNode.name.toLowerCase().includes("mario");

    if (!isPlayer) return;
    if (this.used) return;

    if (this.playerHitFromBottom(otherNode)) {
        cc.log("Mario hit question block from bottom");
        this.activateBlock();
    } else {
        cc.log("Question block touched, but not from bottom");
    }
}
},

```

```

playerHitFromBottom(playerNode) {
    let playerBox = playerNode.getBoundingBoxToWorld();
    let blockBox = this.node.getBoundingBoxToWorld();

    let playerLeft = playerBox.x;
    let playerRight = playerBox.x + playerBox.width;
    let playerCenterY = playerBox.y + playerBox.height / 2;

    let blockLeft = blockBox.x;
    let blockRight = blockBox.x + blockBox.width;
    let blockCenterY = blockBox.y + blockBox.height / 2;

    let playerRb = playerNode.getComponent(cc.RigidBody);

    let playerMovingUp = true;

    if (playerRb) {
        playerMovingUp = playerRb.linearVelocity.y > -20;
    }

    let playerIsBelowBlock = playerCenterY < blockCenterY;

    let horizontalTolerance = 10;

    let playerOverlapsBlockHorizontally =
        playerRight >= blockLeft - horizontalTolerance &&
        playerLeft <= blockRight + horizontalTolerance;

    return (
        playerMovingUp &&
        playerIsBelowBlock &&
        playerOverlapsBlockHorizontally
    );
},

```

```

activateBlock() {
    if (this.used) return;

    this.used = true;

```




```

cc.log("QUESTION BOX ACTIVATED");

this.changeToUsedBlock();

if (this.rewardType === "coin") {
    this.spawnCoin();
} else {
    this.spawnMushroom();
}

this.bumpBlock();

if (window.GameManager && window.GameManager.playPowerUpAppearSound) {
    if (this.rewardType === "mushroom") {
        window.GameManager.playPowerUpAppearSound();
    }
}
},

changeToUsedBlock() {
    if (this.anim) {
        this.anim.stop();
    }

    if (this.sprite && this.usedSpriteFrame) {
        this.sprite.spriteFrame = this.usedSpriteFrame;
        cc.log("Question block changed to used sprite");
    } else {
        cc.log("Used sprite frame is not assigned");
    }
},

bumpBlock() {
    this.node.y = this.originalY + this.bumpHeight;

    this.scheduleOnce(function () {
        this.node.y = this.originalY;
    }, this.bumpTime);
},

spawnMushroom() {
    cc.log("spawnMushroom function started");

    if (!this.mushroomPrefab) {
        cc.log("MUSHROOM PREFAB IS NOT ASSIGNED");
        return;
    }

    let mushroom = cc.instantiate(this.mushroomPrefab);

    let spawnParent = this.getSpawnParent();

    spawnParent.addChild(mushroom);

    if (this.player) {
        this.setGroupRecursive(mushroom, this.player.layer.group);
    }
}

```



```

        cc.v2(0, this.spawnOffsetY)
    );

    let blockLocalPos = spawnParent.convertToNodeSpaceAR(blockWorldPos);

    mushroom.setPosition(blockLocalPos);

    mushroom.active = true;
    mushroom.opacity = 255;
    mushroom.zIndex = 999;
    mushroom.setSiblingIndex(999);

    cc.log("MUSHROOM SPAWNED");

    let mushroomScript = mushroom.getComponent("Mushroom");

    if (mushroomScript) {
        mushroomScript.player = this.player;
        mushroomScript.gameManager = this.gameManager;
    } else {
        cc.log("Mushroom prefab has no Mushroom.js script");
    }
},

spawnCoin() {
    cc.log("spawnCoin function started");

    this.addCoinScore();

    if (window.GameManager && window.GameManager.playCoinSound) {
        window.GameManager.playCoinSound();
    }

    if (!this.coinPrefab) {
        cc.log("COIN PREFAB IS NOT ASSIGNED, score still added");
        return;
    }

    let coin = cc.instantiate(this.coinPrefab);

    let spawnParent = this.getSpawnParent();

    spawnParent.addChild(coin);

    if (this.player) {
        this.setGroupRecursive(coin, this.player.group);
    }

    let blockWorldPos = this.node.convertToWorldSpaceAR(
        cc.v2(0, this.spawnOffsetY)
    );

    let blockLocalPos = spawnParent.convertToNodeSpaceAR(blockWorldPos);

    coin.setPosition(blockLocalPos);

    coin.active = true;
    coin.opacity = 255;
    coin.zIndex = 999;

```




```

        coin.setSiblingIndex(999);

        let coinAnim = coin.getComponent(cc.Animation);

        if (coinAnim) {
            coinAnim.play();
        }

        coin.runAction(
            cc.sequence(
                cc.spawn(
                    cc.moveBy(this.coinMoveTime, cc.v2(0, this.coinMoveUpDistance)),
                    cc.fadeOut(this.coinMoveTime)
                ),
                cc.callFunc(function () {
                    coin.destroy();
                })
            )
        );

        cc.log("COIN SPAWNED");
    },

    addCoinScore() {
        if (window.GameManager && window.GameManager.addScore) {
            window.GameManager.addScore(this.coinScore);
            return;
        }

        if (!this.gameManager) {
            cc.log("Game Manager node is not assigned");
            return;
        }

        let gm = this.gameManager.getComponent("GameManager");

        if (gm && gm.addScore) {
            gm.addScore(this.coinScore);
        }
    },

    getSpawnParent() {
        if (this.player && this.player.parent) {
            return this.player.parent;
        }

        return this.node.parent;
    },

    setGroupRecursive(node, groupName) {
        node.group = groupName;

        for (let i = 0; i < node.children.length; i++) {
            this.setGroupRecursive(node.children[i], groupName);
        }
    }
});

```



okay now i want to make the flag that indicates that you cleared the level. like when you cross it, it means you finished and it shows your score. how to do this

Thought for 14s >

Yes. Make the flag a **trigger area**. When Mario touches/crosses it:

```
Flag detects Player
→ calls GameManager.levelClear()
→ stop player movement
→ show Level Clear panel
→ show final score
```



1. Create the flag node

In your scene:

```
World
└─ Flag
```



Add to **Flag** :

```
Sprite
RigidBody
PhysicsBoxCollider
LevelClearFlag.js
```



Set **Flag** RigidBody:

```
Type: Static
Enabled Contact Listener: checked
```



Set **PhysicsBoxCollider** :

```
Sensor: checked
Size: tall rectangle around the flag
```



Example:

```
W: 30
H: 200
```



2. Create the level clear UI

Under `Canvas`, make:

```
Canvas
├── LevelClearPanel
│   ├── ClearTextLabel
│   └── FinalScoreLabel
```

Set `LevelClearPanel`:

```
Active: unchecked
Group: ui
```

Example label text:

```
ClearTextLabel → COURSE CLEAR!
FinalScoreLabel → SCORE: 0
```

3. Create `LevelClearFlag.js`

```
// JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    gameManager: cc.Node
  },

  onLoad() {
    cc.log("LevelClearFlag running");

    let rb = this.getComponent(cc.RigidBody);

    if (rb) {
      rb.enabledContactListener = true;
    }
  },

  onBeginContact(contact, selfCollider, otherCollider) {
    let otherNode = otherCollider.node;

    let isPlayer =
      otherNode.name.toLowerCase().includes("player") ||
      otherNode.name.toLowerCase().includes("mario");

    if (!isPlayer) return;

    cc.log("Player touched flag");
```



```

        let gm = this.getGameManager();

        if (gm) {
            gm.levelClear();
        }
    },

    getGameManager() {
        if (!this.gameManager) {
            cc.log("Game Manager node is not assigned to flag");
            return null;
        }

        let gm = this.gameManager.getComponent("GameManager");

        if (!gm) {
            let components = this.gameManager.getComponents(cc.Component);

            for (let i = 0; i < components.length; i++) {
                if (typeof components[i].levelClear === "function") {
                    gm = components[i];
                    break;
                }
            }
        }

        return gm;
    }
});

```

Then select **Flag** and drag:

Game Manager → Game Manager node



4. Add these properties to `GameManager.js`

Inside `properties`, add:

<> JavaScript

```

levelClearPanel: cc.Node,
finalScoreLabel: cc.Label,
timeBonusMultiplier: 10

```



So your properties include:

<> JavaScript

```

properties: {
    player: cc.Node,

    lifeLabel: cc.Label,
    scoreLabel: cc.Label,

```




```

scoreLabel: cc.Label,
timerLabel: cc.Label,

levelClearPanel: cc.Node,
finalScoreLabel: cc.Label,

maxTime: 120,
fallLimitY: -400,

bigMarioScale: 1.5,
smallMarioScale: 1,

damageCooldown: 1.0,

timeBonusMultiplier: 10,

// your audio properties...
}

```

5. In `onLoad()`, hide the panel

Add this inside `onLoad()`:

```

</> JavaScript
if (this.levelClearPanel) {
    this.levelClearPanel.active = false;
}

```

6. Add this method to `GameManager.js`

Paste this inside your `GameManager.js`:

```

</> JavaScript
levelClear() {
    if (this.gameEnded) return;

    this.gameEnded = true;

    cc.log("LEVEL CLEAR");

    let timeBonus = Math.ceil(this.timer) * this.timeBonusMultiplier;
    let finalScore = this.score + timeBonus;

    this.score = finalScore;
    this.updateUI();

    if (this.levelClearPanel) {
        this.levelClearPanel.active = true;
    }

    if (this.finalScoreLabel) {

```



```

    if (this.finalScoreLabel) {
        this.finalScoreLabel.string = "SCORE: " + finalScore;
    }

    if (window.GameManager && window.GameManager.playLevelClearSound) {
        window.GameManager.playLevelClearSound();
    }

    this.stopPlayer();
}

```

Then add this helper function too:

```

</> JavaScript
stopPlayer() {
    if (!this.player) return;

    let rigidBody = this.player.getComponent(cc.RigidBody);

    if (rigidBody) {
        rigidBody.linearVelocity = cc.v2(0, 0);
    }

    let playerController = this.player.getComponent("PlayerController");

    if (playerController) {
        playerController.enabled = false;
    }
}

```

7. Add level clear sound function if needed

In your `GameManager.js`, add this function near your other sound functions:

```

</> JavaScript
playLevelClearSound() {
    this.playSFX(this.levelClear, "levelClear");
}

```

Make sure you already have this property:

```

</> JavaScript
levelClear: cc.AudioClip,

```

Then drag your `levelClear` audio into the slot.



Prompt: i want mario to jump over the flag. right now the mario stays before the block of the flag.

Answer:

Yes, right now Mario stops because your `levelClear()` calls:

```
</> JavaScript
this.stopPlayer();
```

That sets Mario's velocity to `0`, so he stays before the flag.

Do this instead:

touch flag → disable player control → give Mario jump velocity → show Level Clear UI after short

1. Make sure flag collider is Sensor

Select your `Flag` node:

```
PhysicsBoxCollider
Sensor: checked
```

If `Sensor` is unchecked, Mario will be blocked by the flag.

2. Add these properties to `GameManager.js`

Inside `properties`, add these:

```
</> JavaScript
finishJumpXSpeed: 160,
finishJumpYSpeed: 360,
showLevelClearDelay: 0.8
```

Example:

```
</> JavaScript
timeBonusMultiplier: 10,

finishJumpXSpeed: 160,
finishJumpYSpeed: 360,
showLevelClearDelay: 0.8,
```

3. Replace your `levelClear()` with this

```
</> JavaScript
levelClear() {
  if (this.gameEnded) return;
```



```

    this.gameEnded = true;

    cc.log("LEVEL CLEAR");

    let timeBonus = Math.ceil(this.timer) * this.timeBonusMultiplier;
    let finalScore = this.score + timeBonus;

    this.score = finalScore;
    this.updateUI();

    this.playLevelClearSound();

    // Make Mario jump over the flag
    this.makeMarioJumpOverFlag();

    // Show clear UI after Mario jumps
    this.scheduleOnce(function () {
        this.showLevelClearUI(finalScore);
    }, this.showLevelClearDelay);
},

```

4. Add these two functions

Paste these inside `GameManager.js` :

```

<> JavaScript
makeMarioJumpOverFlag() {
    if (!this.player) return;

    // Disable player input
    let playerController = this.player.getComponent("PlayerController");

    if (playerController) {
        playerController.enabled = false;
    }

    // Make Mario jump forward
    let rigidBody = this.player.getComponent(cc.RigidBody);

    if (rigidBody) {
        rigidBody.linearVelocity = cc.v2(
            this.finishJumpXSpeed,
            this.finishJumpYSpeed
        );
    }
},

showLevelClearUI(finalScore) {
    if (this.LevelClear) {
        this.LevelClear.active = true;
    }

    if (this.LevelClearLabel) {
        this.LevelClearLabel.string = "LEVEL CLEAR!";
    }
}

```



```

    }

    if (this.FinalScoreLabel) {
        this.FinalScoreLabel.string = "SCORE: " + finalScore;
    }

    // Stop Mario after showing result
    this.stopPlayer();
},

```

5. Keep your `stopPlayer()` function

Your current one is fine:

```

</> JavaScript
stopPlayer() {
    if (!this.player) return;

    let rigidBody = this.player.getComponent(cc.RigidBody);

    if (rigidBody) {
        rigidBody.linearVelocity = cc.v2(0, 0);
        rigidBody.angularVelocity = 0;
    }

    let playerController = this.player.getComponent("PlayerController");

    if (playerController) {
        playerController.enabled = false;
    }
},

```

Now the result should be:

```

Mario touches flag
Level clear triggers
Mario jumps forward over the flag
After 0.8 seconds, LevelClear UI appears with final score

```

If Mario jumps too low, increase:

```

</> JavaScript
finishJumpYSpeed: 420

```

If Mario does not move far enough forward, increase:

```

</> JavaScript
finishJumpXSpeed: 220

```

Prompt: i want it when the player touch the ground, it'll change into player_walk even before i press to walk

Answer:

Thought for 12s >

Yes, change the grounded/no-movement state to `player_walk` instead of stopping the animation.

Replace your full `PlayerController.js` with this version:

```
JavaScript

cc.Class({
  extends: cc.Component,

  properties: {
    moveSpeed: 120,
    jumpForce: 520,

    walkAnimName: "player_walk",
    jumpAnimName: "player_jump",

    jumpAnimLockTime: 0.25
  },

  onLoad() {
    this.moveDir = 0;
    this.currentAnim = "";

    this.isGrounded = false;
    this.groundContactCount = 0;
    this.jumpAnimTimer = 0;

    cc.director.getPhysicsManager().enabled = true;
    cc.director.getPhysicsManager().gravity = cc.v2(0, -640);

    this.rigidBody = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    if (this.rigidBody) {
      this.rigidBody.enabledContactListener = true;
      this.rigidBody.fixedRotation = true;
    }

    this.playAnim(this.walkAnimName);

    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  onDestroy() {
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN, this.onKeyDown, this);
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP, this.onKeyUp, this);
  },

  update(dt) {
    this.node.x += this.moveDir * this.moveSpeed * dt;

    if (this.jumpAnimTimer > 0) {
      this.jumpAnimTimer -= dt;
    }
  }
});
```



```

        this.flipPlayer();
        this.updateAnimation();
    },

    onKeyDown(event) {
        if (event.keyCode === cc.macro.KEY.a || event.keyCode === cc.macro.KEY.left) {
            this.moveDir = -1;
        }

        if (event.keyCode === cc.macro.KEY.d || event.keyCode === cc.macro.KEY.right) {
            this.moveDir = 1;
        }

        if (
            event.keyCode === cc.macro.KEY.space ||
            event.keyCode === cc.macro.KEY.w ||
            event.keyCode === cc.macro.KEY.up
        ) {
            cc.log("JUMP KEY PRESSED");
            this.jump();
        }
    },

    onKeyUp(event) {
        if (
            event.keyCode === cc.macro.KEY.a ||
            event.keyCode === cc.macro.KEY.left ||
            event.keyCode === cc.macro.KEY.d ||
            event.keyCode === cc.macro.KEY.right
        ) {
            this.moveDir = 0;
        }
    },

    jump() {
        if (!this.rigidBody) return;

        if (!this.isGrounded) {
            cc.log("Cannot jump because player is not grounded");
            return;
        }

        let velocity = this.rigidBody.linearVelocity;
        velocity.y = this.jumpForce;
        this.rigidBody.linearVelocity = velocity;

        this.isGrounded = false;
        this.groundContactCount = 0;

        this.jumpAnimTimer = this.jumpAnimLockTime;
        this.playAnim(this.jumpAnimName);

        cc.log("Playing jump animation: " + this.jumpAnimName);

        if (window.GameManager && window.GameManager.playJumpSound) {
            window.GameManager.playJumpSound();
        } else if (window.SoundManager && window.SoundManager.playJump) {
            window.SoundManager.playJump();
        }
    }
}

```



```

},

flipPlayer() {
  if (this.moveDir < 0) {
    this.node.scaleX = -Math.abs(this.node.scaleX);
  } else if (this.moveDir > 0) {
    this.node.scaleX = Math.abs(this.node.scaleX);
  }
},

updateAnimation() {
  if (!this.rigidBody) return;

  let velocity = this.rigidBody.linearVelocity;

  if (this.jumpAnimTimer > 0) {
    this.playAnim(this.jumpAnimName);
    return;
  }

  if (!this.isGrounded || Math.abs(velocity.y) > 15) {
    this.playAnim(this.jumpAnimName);
    return;
  }

  // Player touched the ground, so go back to player_walk.
  this.playAnim(this.walkAnimName);
},

playAnim(animName) {
  if (!this.anim) {
    cc.log("No Animation component on Player");
    return;
  }

  if (this.currentAnim === animName) return;

  let state = this.anim.getAnimationState(animName);

  if (!state) {
    cc.log("Animation clip not found: " + animName);
    return;
  }

  this.currentAnim = animName;
  this.anim.play(animName);
},

onBeginContact(contact, selfCollider, otherCollider) {
  let otherNode = otherCollider.node;

  if (this.isGroundObject(otherNode)) {
    this.groundContactCount += 1;
    this.isGrounded = true;

    // Important: stop forcing jump animation once player lands.
    this.jumpAnimTimer = 0;
  }
}

```

```

        this.playAnim(this.walkAnimName);
    },

    onEndContact(contact, selfCollider, otherCollider) {
        let otherNode = otherCollider.node;

        if (this.isGroundObject(otherNode)) {
            this.groundContactCount -= 1;

            if (this.groundContactCount <= 0) {
                this.groundContactCount = 0;
                this.isGrounded = false;
            }
        }
    },

    isGroundObject(node) {
        if (!node) return false;

        let name = node.name.toLowerCase();

        return (
            name.includes("ground") ||
            name.includes("block") ||
            name.includes("wall") ||
            name.includes("pipe") ||
            name.includes("tube") ||
            name.includes("question")
        );
    }
});

```

The key fix is here:

```

</> JavaScript
this.jumpAnimTimer = 0;
this.playAnim(this.walkAnimName);

```

inside `onBeginContact()`, so once Mario touches the ground, it immediately changes back to `player_walk`.

📄 👍 💬 📁 ↺ ...

